QUICHOTTE

QUantum Interacting CHains: Out-of-equilibrium TransporT

User manual

Aadi Konidena, King's College London

October 2026

Abstract

QUICHOTTE computes the spin-resolved current–voltage characteristics of a helical molecule with spin–orbit coupling (SOC) and an on-site Hubbard interaction, coupled to two leads. It uses self-consistent non-equilibrium Green's functions (NEGF), with the interaction treated to zeroth, first (Hartree–Fock) or second order in U. The code was written to study chirality-induced spin selectivity (CISS). This manual gives the model, the equations the code solves, the numerical methods, a complete reference for the input and output files, and the practical workflow. The companion code *SANCHO* (in `SANCHO/`) treats a spinless chain to third order in U and has its own manual.

Contents

1 Quick start **3**

2 Model **3**

2.1 Units and conventions . . . 3

2.2 Basis . . . 3

2.3 Helix geometry . . . 3

2.4 Hamiltonian . . . 4

2.5 Leads . . . 4

3 Non-equilibrium Green's functions **4**

3.1 Energy grid . . . 4

3.2 Dyson and Keldysh equations . . . 5

3.3 Equal-time quantities . . . 5

3.4 Current . . . 5

4 Interaction self-energies **5**

4.1 First order (Hartree–Fock), `order` ≥ 1 . . . 5

4.2 Second order, `order` $= 2$ . . . 6

4.3 Total self-energy . . . 6

5 Self-consistency **6**

5.1 The fixed-point problem . . . 6

5.2 Modified Broyden mixing . . . 6

5.3 Convergence . . . 7

5.4 Sanity checks . . . 7

5.5 Voltage sweep and starting guess . . . 7

6 Input reference: `input.dat` **7**

7 Output reference **8**

8 Workflow **9**

8.1 Runs on a cluster . . . 9

8.2 Parallelism . . . 9

8.3 Tools . . . 9

9 Using your own Hamiltonian **10**

9.1 Your own model in three steps . . . 10

9.2 What the matrix must look like . . . 11

9.3 Same terms, different shape: `hamiltonian = xyz` . . . 11

9.4 Several models of your own . . . 12

10 Code structure **12**

10.1 Common changes . . . 13

11 Testing and validation **13**

12 Known limitations **13**

1 Quick start

```
git clone https://github.com/Aadik1/QUICHOTTE.git && cd QUICHOTTE
make            # builds bin/quichotte.x (needs gfortran, BLAS/LAPACK, FFTW3)
make test       # 5 regression tests, a few seconds; all must PASS

mkdir -p ~/runs/first && cd ~/runs/first
cp ~/QUICHOTTE/examples/helix_4x2_order2/input.dat .     # edit as needed
OMP_NUM_THREADS=4 ~/QUICHOTTE/bin/quichotte.x
```

Each run lives in its own directory, *outside* the repository, which holds only its `input.dat`. All results are written to that directory (Section 7).

Build requirements.

platform	build command	needs
macOS	`make`	`brew install gcc openblas fftw`
Linux	`make`	`gfortran`, `libopenblas-dev`, `libfftw3-dev`
KCL CREATE	`make PLATFORM=create`	`source hpc/create/modules.sh`
ARCHER2	`make PLATFORM=archer2`	`source hpc/archer2/modules.sh`

Compiler flags and library paths for each platform are in `make/<platform>.mk`. Any Fortran 2008 compiler with OpenMP should work.

2 Model

2.1 Units and conventions

Energies are in eV, voltages in V and temperatures in K. The code uses $\hbar = e = 1$ (`hbar` $= 1$ in `common/constants.f90`); the factors of $\hbar$ are kept in the equations below and in the source, so that they can be restored. $k_B = 8.6173303 \times 10^{-5}$ eV/K.

2.2 Basis

The molecule has $N_{\text{sites}} = N_{\text{ions}}\, N_{\text{turns}}$ sites with one orbital each. Every site carries two spin-orbitals, numbered

$$2i - 1 = (i, \uparrow), \qquad 2i = (i, \downarrow), \qquad i = 1, \ldots, N_{\text{sites}},$$

so all matrices have dimension $N_{\text{orb}} = 2N_{\text{sites}}$ (`Norb` in the code). The spin quantisation axis is z, the helix axis.

2.3 Helix geometry

Site i sits at (`helix_coordinate` in `src/hamiltonian.f90`)

$$\mathbf{r}_i = \Big(R\cos\phi_i,\ R\sin\phi_i,\ (i-1)\,\Delta z\Big), \qquad \phi_i = h\,\frac{2\pi(i-1)}{N_{\text{ions}}}, \qquad \Delta z = \frac{L}{N_{\text{turns}}(N_{\text{ions}}-1)}, \quad (1)$$

with $R =$ `hel_radius`, $L =$ `hel_length`, and $h = \pm 1 =$ `hand` (right/left-handed). For a single turn L is exactly the end-to-end length; for N_{turns} turns the end-to-end length is $L\,(N_{\text{sites}} - 1)/\big(N_{\text{turns}}(N_{\text{ions}} - 1)\big)$, so the helix becomes slightly longer than L (for example 5.83 for $L = 5$, $N_{\text{ions}} = 4$, $N_{\text{turns}} = 2$). Only unit vectors along bonds enter the Hamiltonian, so the SOC depends on the *shape* of the helix (the ratio of Δz to R), not on its overall size.

2.4 Hamiltonian

$$H = \sum_{i\sigma} \varepsilon\, c^\dagger_{i\sigma} c_{i\sigma} + t \sum_{i\sigma} \big(c^\dagger_{i\sigma} c_{i+1,\sigma} + \text{h.c.}\big) + \sum_{i} \big(c^\dagger_{i}\, \mathcal{V}_{i,i+2}\, c_{i+2} + \text{h.c.}\big) + \sum_{i} U_i\, n_{i\uparrow} n_{i\downarrow}, \tag{2}$$

with $\varepsilon =$ `E_CC`, $t =$ `t_hop`, and the next-nearest-neighbour spin–orbit term (a 2×2 matrix in spin space)

$$\mathcal{V}_{i,i+2} = i\lambda\, \big(\hat{\mathbf{d}}_{i,i+1} \times \hat{\mathbf{d}}_{i,i+2}\big) \cdot \boldsymbol{\sigma}, \qquad \hat{\mathbf{d}}_{i,j} = \frac{\mathbf{r}_i - \mathbf{r}_j}{|\mathbf{r}_i - \mathbf{r}_j|}, \tag{3}$$

where $\lambda =$ `lamb` and $\boldsymbol{\sigma}$ are the Pauli matrices (`soc_matrix_elements`). The non-interacting part is built by `build_helix_hamiltonian`; the Hubbard term enters only through the self-energy (Section 4). $U_i =$ `U_int` on every site, or one value per site from `hubbard_file`.

This is the helix model built by `build_helix_hamiltonian`. Nothing else in the code depends on it: any other single-particle Hamiltonian can be used instead, as described in Section 9.

2.5 Leads

Both leads are in the wide-band limit. The left lead couples to site 1 with a spin-dependent strength, and the right lead couples to the last site:

$$\Gamma_L = \text{diag}(\Gamma_{L\uparrow}, \Gamma_{L\downarrow}, 0, \ldots, 0), \qquad \Gamma_R = \text{diag}(0, \ldots, 0, \Gamma_R, \Gamma_R), \tag{4}$$

($\Gamma_{L\uparrow} =$ `GammaL_up`, $\Gamma_{L\downarrow} =$ `GammaL_down`, $\Gamma_R =$ `GammaR`). Their retarded self-energy is $\Sigma^r_{\text{leads}} = -\frac{i}{2}(\Gamma_L + \Gamma_R)$. The bias is applied to the *left* lead only:

$$f_L(\omega) = \frac{1}{e^{\beta(\omega-\mu+V)} + 1}, \qquad f_R(\omega) = \frac{1}{e^{\beta(\omega-\mu)} + 1}, \qquad \beta = 1/k_B T, \tag{5}$$

i.e. $\mu_L = \mu - V$, $\mu_R = \mu$ (`fermi` in `src/green_functions.f90`). Because the bias is asymmetric, the I–V curve need not be odd in V once $U \neq 0$.

3 Non-equilibrium Green's functions

3.1 Energy grid

All functions of energy are stored on the uniform grid

$$\omega_k = \omega_{\text{init}} + k\,\delta, \qquad k = 1, \ldots, N_\omega, \qquad \omega_{\text{init}} = \epsilon_{\min} - \texttt{dw}, \qquad N_\omega = \left\lfloor \frac{\epsilon_{\max} + \texttt{up} - \omega_{\text{init}}}{\delta} \right\rfloor, \tag{6}$$

where $\epsilon_{\min,\max}$ are the extreme eigenvalues of H and $\delta =$ `delta`. The margins `dw`, `up` must contain the broadened spectrum (the sanity check of Section 5.4 warns if spectral weight is missing).

3.2 Dyson and Keldysh equations

At every grid energy (`interacting_gf` in `src/scf.f90`):

$$G^r(\omega) = \Big[\hbar\omega - H + \tfrac{i}{2\hbar}(\Gamma_L+\Gamma_R) - \Sigma^r(\omega)\Big]^{-1}, \qquad G^a(\omega) = \big[G^r(\omega)\big]^\dagger, \tag{7}$$

$$G^<(\omega) = G^r(\omega)\Big[\tfrac{i}{\hbar}\big(f_L(\omega)\Gamma_L + f_R(\omega)\Gamma_R\big) + \Sigma^<(\omega)\Big]G^a(\omega), \tag{8}$$

$$G^>(\omega) = G^<(\omega) + G^r(\omega) - G^a(\omega). \tag{9}$$

The non-interacting functions G_0 (`noninteracting_gr_ga`, `noninteracting_gl_gg`) are the same equations with $\Sigma = 0$.

3.3 Equal-time quantities

$$G^<(t{=}0) = \frac{\delta}{2\pi}\sum_k G^<(\omega_k), \qquad n_{i\sigma} = -i\hbar\, G^<_{i\sigma,i\sigma}(t{=}0), \qquad \langle S_\alpha\rangle_i = -\frac{i\hbar^2}{2}\,\mathrm{Tr}_\sigma\big[\sigma_\alpha\, G^<_{ii}(t{=}0)\big], \tag{10}$$

where $G^<_{ii}$ is the 2×2 spin block of site i (`equal_time_lesser`, `spin_texture`).

3.4 Current

The spin-resolved particle current through the left lead is (`Current`, `current_integrand` in `src/observables.f90`)

$$J_\sigma = \frac{1}{\hbar}\frac{\delta}{2\pi}\sum_k \mathrm{Re}\,\mathrm{Tr}_\sigma\Big\{i\,\Gamma_L\Big[\big(f_L(\omega_k)-1\big)G^<(\omega_k) - f_L(\omega_k)\,G^>(\omega_k)\Big]\Big\}, \tag{11}$$

where Tr_σ sums over the spin-σ orbitals only. Using Eq. (9), the bracket is $-\big[G^< + f_L(G^r - G^a)\big]$, so this is minus the Meir–Wingreen expression $\frac{i}{\hbar}\int\frac{d\omega}{2\pi}\,\mathrm{Tr}\{\Gamma_L[G^< + f_L(G^r-G^a)]\}$. With this sign the current is positive for $V>0$. The output also gives $J = J_\uparrow + J_\downarrow$; the spin polarisation is $J_\uparrow - J_\downarrow$.

4 Interaction self-energies

All self-energies are functionals of the *input* Green's functions of the current SCF step, G_{in} (Section 5), and are evaluated in `src/self_energies.f90`.

4.1 First order (Hartree–Fock), `order` ≥ 1

The first-order term is static and local to each site. With $\rho_i = -i\hbar\,G^<_{ii}(t{=}0)$ the 2×2 density matrix of site i,

$$\Sigma^{(1)}_{i\sigma,i\sigma'} = i\hbar\,U_i\Big[G^<_{i\sigma,i\sigma'}(0) - \delta_{\sigma\sigma'}\,\mathrm{Tr}\,G^<_{ii}(0)\Big] \;=\; U_i\big[\delta_{\sigma\sigma'}\,\mathrm{Tr}\,\rho_i - \rho_{i,\sigma\sigma'}\big], \tag{12}$$

i.e. $\Sigma_{\uparrow\uparrow} = Un_\downarrow$, $\Sigma_{\downarrow\downarrow} = Un_\uparrow$ (Hartree) and $\Sigma_{\uparrow\downarrow} = -U\rho_{\uparrow\downarrow}$ (Fock). It has no lesser component (`sigma_first_order`).

4.2 Second order, `order` = 2

The second-order diagrams couple every pair of sites (i, j). Writing G_{ij} for the 2×2 spin block between sites i and j, and working in the time domain (the variable conjugate to ω on the grid):

$$\Sigma^{(2)r}_{ij}(t) = \hbar^2 U_i U_j \Big\{ G^r_{ij}(t)\ \mathrm{adj}\big[G^<_{ji}(-t)\, G^>_{ij}(t)\big] + G^<_{ij}(t)\ \mathrm{adj}\big[G^<_{ji}(-t)\, G^r_{ij}(t) + G^a_{ji}(-t)\, G^<_{ij}(t)\big] \Big\}, \tag{13}$$

$$\Sigma^{(2)<}_{ij}(t) = \hbar^2 U_i U_j\ G^<_{ij}(t)\ \mathrm{adj}\big[G^>_{ji}(-t)\, G^<_{ij}(t)\big], \tag{14}$$

with the 2×2 adjugate $\mathrm{adj}\, M = (\mathrm{Tr}\, M)\mathbb{1} - M$, i.e. direct minus exchange (`adj2`). Each product of three functions is a double sum over the grid in energy, $\sum_{k_1,k_2} A(\omega_{k_1})B(\omega_{k_2})C(\omega - \omega_{k_1} + \omega_{k_2})$, weighted by $(\delta/2\pi)^2$.

FFT evaluation. QUICHOTTE evaluates these sums for all energies at once by fast Fourier transforms (`sigma2_compute`, FFTW). Each element of G^r and $G^<$ is zero-padded to length $N_f \geq 2N_\omega - 1$ (the smallest such length with prime factors $2, 3, 5, 7$), transformed, multiplied as above, and transformed back. The zero padding makes the result *identical* to the direct discrete sums, so it is not an approximation. Only G^r and $G^<$ are transformed; $G^a_{ij}(t) = [G^r_{ji}(-t)]^\dagger$ and $G^> = G^< + G^r - G^a$ follow exactly. The cost per SCF step is $\mathcal{O}(N^2_{\mathrm{sites}}\, N_f \log N_f)$ instead of $\mathcal{O}(N^2_{\mathrm{sites}} N^2_\omega)$.

4.3 Total self-energy

$$\Sigma^r = \Sigma^{(1)} + \Sigma^{(2)r}, \qquad \Sigma^< = \Sigma^{(2)<}, \tag{15}$$

including only the terms up to `order` (`sigma_retarded`, `sigma_lesser`).

5 Self-consistency

5.1 The fixed-point problem

At each voltage the code solves $G = \mathcal{F}[G]$, where $\mathcal{F}$ builds $\Sigma[G]$ and then applies Eqs. (7)–(9). G^r and $G^<$ on the whole grid are packed into one real vector x (`pack_gf` in `common/linear_algebra.f90`). Per energy the vector holds

$$\big[\mathrm{Re}\, G^r_{ab}\ (N^2_{\mathrm{orb}})\ \big|\ \mathrm{Im}\, G^r_{ab}\ (N^2_{\mathrm{orb}})\ \big|\ \mathrm{Re}\, G^<_{a<b}\ \big|\ \mathrm{Im}\, G^<_{a\leq b}\big].$$

Only the independent half of $G^<$ is stored, because $G^<$ is anti-Hermitian, $G^<_{ba} = -\big(G^<_{ab}\big)^*$, so its diagonal is purely imaginary. The residual is $F(x) = x_{\mathrm{out}}(x) - x$ (`scf_residual`).

5.2 Modified Broyden mixing

$F(x) = 0$ is solved by Johnson's modified Broyden method (D. D. Johnson, Phys. Rev. B **38**, 12807 (1988); `common/broyden_mixing.f90`). With $\alpha =$ `alpha_mix`,

$$\Delta F_k = \frac{F_k - F_{k-1}}{\|F_k - F_{k-1}\|}, \qquad u_k = \alpha\, \Delta F_k + \frac{x_k - x_{k-1}}{\|F_k - F_{k-1}\|}, \tag{16}$$

$$\gamma = \big(w_0^2 \mathbb{1} + \Delta F^T \Delta F\big)^{-1} \Delta F^T F_n, \qquad w_0 = 0.01, \tag{17}$$

$$x_{n+1} = x_n + \alpha F_n - \sum_k \gamma_k\, u_k. \tag{18}$$

The last `nhist` pairs $(\Delta F_k, u_k)$ are kept in a ring buffer (`nhist` $= 0$ gives plain linear mixing). No Jacobian is stored, so each step costs one evaluation of F plus $\mathcal{O}(n \cdot \texttt{nhist})$ BLAS work, with n the vector length.

5.3 Convergence

A voltage is converged when $\max_m |F_m| < \texttt{tolf}$. This is tested *before* mixing, so the stored solution is an input whose own output differs from it by less than `tolf` in every component. At most `maxscf` steps are taken. The history of every voltage is in `scf_V_<V>.dat`.

5.4 Sanity checks

After every step (`common/scf_checks.f90`), for every spin-orbital a:

$$n_a = -i\hbar \tfrac{\delta}{2\pi} \textstyle\sum_k G^{<}_{aa}(\omega_k), \qquad W_a = -\hbar \tfrac{\delta}{\pi} \textstyle\sum_k \operatorname{Im} G^{r}_{aa}(\omega_k).$$

- **Runaway** (NaN, or $n_a \notin [-1, 2]$): the SCF for this voltage is abandoned.
- **Unphysical** ($n_a \notin [-0.05, 1.05]$ or $W_a < 0.5$): a converged solution of this kind is *not* accepted. $W_a < 1$ usually means the grid margins `dw`, `up` are too small.

5.5 Voltage sweep and starting guess

Voltages run from `V` to `Vf` in steps of `delv`. The first voltage starts from G_0. With `continuation = T` (the default), each later voltage starts from the converged G of the previous one, which usually saves many iterations; with `continuation = F` every voltage starts from G_0. A voltage that fails (not converged, runaway or unphysical) is *flagged* (last column = 0 in `Volt_Current`), the sweep carries on, and the next voltage starts from G_0 again. If any voltage failed, the program exits with code 3.

Comparing a sweep with `continuation = T` against one with `continuation = F` (every voltage from G_0) is a quick check for hysteresis.

6 Input reference: `input.dat`

One `key = value` per line, in any order; `#` starts a comment. Keys are case-sensitive. The reader (`common/input_parser.f90`):

- stops with a list of *all* missing required keys and duplicated keys;
- warns about keys it does not know (usually a typo);
- reports the line number of a value it cannot read.

If there is no `input.dat` it reads `inputSOC.dat` (the old name). Old key names (third column) are still accepted. `examples/helix_4x2_order2/input.dat` documents every key.

key	default	old name	meaning
`V, Vf, delv`	required		first voltage, last voltage, step [V]
`T`	required		temperature of both leads [K]

key	default	old name	meaning
mu	required		chemical potential at $V = 0$ [eV]
hamiltonian	helix		model: helix, xyz or custom (Section 9)
xyz_file	molecule.xyz		site positions, for hamiltonian = xyz
N_sites	from model		number of sites; required for custom, must agree with the model otherwise
N_ions	helix only		sites per turn (≥ 2)
N_turns	helix only		number of turns
hel_length	helix only		L, see the helix geometry above
hel_radius	helix only		R
hand	helix only		$+1$ right-handed, -1 left-handed
E_CC	required		on-site energy ε [eV]
t_hop	required		nearest-neighbour hopping t [eV]
lamb	helix, xyz		SOC strength λ [eV]
U_int	required	Hubbard	uniform U [eV]
hubbard_file	none		file with one U per site (overrides U_int)
order	required		0 non-interacting, 1 Hartree–Fock, 2 second order
GammaL_up	required		$\Gamma_{L\uparrow}$ [eV]
GammaL_down	required		$\Gamma_{L\downarrow}$ [eV]
GammaR	required	GammaR_both	Γ_R, both spins [eV]
dw, up	required		grid margins below/above the spectrum of H [eV]
delta	required		grid spacing δ [eV]
alpha_mix	0.5	pulay	Broyden linear-mixing weight α
nhist	16		Broyden history length (0 = linear mixing)
maxscf	300		maximum SCF steps per voltage
tolf	10^{-6}		convergence threshold on $\max \lvert F \rvert$
continuation	T		start each voltage from the previous one
restart	F		accepted for old files, not used

Choosing the numerical parameters.

- delta must resolve the narrowest feature of the spectral function, whose width is set by Γ and U. Halve it and compare.
- dw, up must contain the broadened spectrum; check that $W_a \approx 1$.
- Runtime and memory grow as $N_\omega \propto (\text{bandwidth} + \texttt{dw} + \texttt{up})/\delta$. The Broyden history needs $2\,\texttt{nhist} \cdot n$ doubles, with $n \approx 3N_{\text{orb}}^2 N_\omega$.
- If a voltage does not converge, lower alpha_mix (e.g. 0.2) or raise maxscf; large $U/|t|$ needs smaller α.

7 Output reference

file	content
Volt_Current_<order>.dat	one line per voltage: V, $J_\uparrow$, $J_\downarrow$, J, converged (1/0)
populations.dat	one line per site per voltage: V, site, $\langle S_x \rangle$, $\langle S_y \rangle$, $\langle S_z \rangle$, $n_\downarrow$, $n_\uparrow$, n

file	content
spectral_function.dat	V, ω, $A_{aa}(\omega) = -2\,\mathrm{Im}\,G^r_{aa}$ for every spin-orbital; one block per voltage (`index k` in gnuplot)
scf_V_<V>.dat	SCF history at that voltage: step, max $\lvert F\rvert$, rms$\lvert F\rvert$, history length used
info_Hamiltonian.dat	H, a Hermiticity check, eigenvalues, eigenvectors
runtime_datasheet.dat	code version (`git describe`), voltages processed, wall time

Exit codes. 0: every voltage converged. 1: bad input (the message says why). 3: finished, but at least one voltage failed (column 5 = 0).

Version stamp. The Makefile writes the output of `git describe --always --dirty --tags` into the binary. It is printed at start-up and in `runtime_datasheet.dat`, so every result records the code that produced it. Tag the versions you use for papers (`git tag paper1-v1`).

8 Workflow

8.1 Runs on a cluster

`hpc/<cluster>/job.*` run QUICHOTTE in the directory they are submitted from. Set `EXE` at the top of the script once:

```
cd /scratch/users/$USER/R2G2D
cp ~/QUICHOTTE/examples/helix_4x2_order2/input.dat .
sbatch ~/QUICHOTTE/hpc/create/job.slurm          # CREATE (Slurm)
qsub   ~/QUICHOTTE/hpc/young/job.sge             # Young (SGE)
sbatch $WORK/QUICHOTTE/hpc/archer2/job.slurm  # ARCHER2 (under /work)
```

The scripts set `OMP_NUM_THREADS` to the allocated cores, `OPENBLAS_NUM_THREADS=1` (the LAPACK calls already sit inside OpenMP loops) and `OMP_STACKSIZE=512M`.

8.2 Parallelism

OpenMP parallelises the loop over energies (Dyson/Keldysh) and the FFT self-energy over site pairs. One run uses one node; scale `OMP_NUM_THREADS` up to the number of cores.

8.3 Tools

`make tools` builds:

- `bin/helix_xyz.x [input.dat]`: writes `helix.xyz` with the site positions of the run (for visualisation, or as a starting point for `build_hamiltonian_from_xyz`).
- `bin/dIdV.x [file] [column]`: differential conductance dJ/dV by central differences, written to `dI_dV.dat` (column 4 = J).

9 Using your own Hamiltonian

The helix of Section 2 is only one model. Nothing else in QUICHOTTE depends on it: the Green's functions, self-energies, SCF loop and observables see nothing but the matrix H. The input key `hamiltonian` chooses which model builds H:

`hamiltonian =`	routine in `src/hamiltonian.f90`	number of sites
`helix (default)`	`build_helix_hamiltonian`	`N_ions`×`N_turns`
`xyz`	`build_hamiltonian_from_xyz`	atoms in `xyz_file`
`custom`	`build_custom_hamiltonian`	the key `N_sites`

The dispatcher `build_hamiltonian` calls the chosen routine, so the main program never needs editing. `N_sites` may also be given for `helix` and `xyz`, in which case it must agree with the model.

9.1 Your own model in three steps

1. Edit the body of `build_custom_hamiltonian`, the last routine in `src/hamiltonian.f90`. As shipped it is a plain spin-degenerate chain (on-site `E_CC`, hopping `t_hop`, no SOC), so it runs straight away:

```
subroutine build_custom_hamiltonian()
   integer :: i, ii

   H = (0.d0, 0.d0)
   do i = 1, N_sites
      ii = 2*i                        !...ii-1 = (i,up), ii = (i,down)
      H(ii - 1, ii - 1) = E_CC        !...on-site energy, spin up
      H(ii, ii) = E_CC                !...on-site energy, spin down
      if (i .lt. N_sites) then        !...hop i -> i+1, spin conserving
         H(ii - 1, ii + 1) = t_hop    !...up   -> up
         H(ii, ii + 2) = t_hop        !...down -> down
      end if
   end do
   call make_hermitian()              !...upper triangle -> whole matrix
end subroutine build_custom_hamiltonian
```

   All run parameters (`N_sites`, `E_CC`, `t_hop`, `lamb`, ...) are available inside the routine. New parameters: declare them in `src/parameters.f90` and read them in `read_input` (`src/input_output.f90`) with `call input_get('my_key', my_var)`; add `default=...` to make a key optional.

2. In `input.dat` set `hamiltonian = custom` and `N_sites =` *number of sites*. The helix keys (`N_ions`, `N_turns`, `hel_length`, `hel_radius`, `hand`) and `lamb` are then optional.

3. Build (`make`) and check one run. `info_Hamiltonian.dat` should show your H, all-zero Hermiticity differences and the expected eigenvalues. With `order = 0`, the current at $V = 0$ should be zero. Then run `make test`, and add a small test case for your model (Section 11) so that later changes cannot break it unnoticed.

9.2 What the matrix must look like

1. **Storage.** H is the module variable `H` of `src/hamiltonian.f90`, declared `complex(dp), allocatable :: H(:,:)`. The main program allocates it as `H(Norb, Norb)`, with $N_{\mathrm{orb}} = 2N_{\mathrm{sites}}$, before your routine is called; your routine only fills it (do not reallocate it).

2. **Basis order (essential).** Spin-orbitals are interleaved by site:

$$2i-1 = (i,\uparrow), \qquad 2i = (i,\downarrow), \qquad i = 1,\ldots,N_{\mathrm{sites}}.$$

   The rest of the code relies on this order. The Hubbard U acts on the 2×2 spin block of each site, and `U_site` is stored on the spin-up index. The first- and second-order self-energies loop over sites in steps of two. The spin texture, the populations and the spin-resolved current read odd indices as spin up and even indices as spin down. The lead couplings $\Gamma_{L,R}$ use the same order. A Hamiltonian written as two spin blocks, $\begin{pmatrix} H_{\uparrow\uparrow} & H_{\uparrow\downarrow} \\ H_{\downarrow\uparrow} & H_{\downarrow\downarrow} \end{pmatrix}$, must be reordered: element $(H_{\sigma\sigma'})_{ij}$ goes to `H(2i-1+s, 2j-1+s')` with $s, s' = 0$ for $\uparrow$ and 1 for $\downarrow$.

3. **Hermitian.** $H = H^{\dagger}$. The easiest way is to fill only the *upper* triangle (column $\geq$ row, diagonal included) and finish with `call make_hermitian()`, as the template does. This overwrites the lower triangle with the conjugate of the upper one (and makes the diagonal real), so anything you wrote below the diagonal is lost. `info_Hamiltonian.dat` lists $H_{ij} - H_{ji}^{*}$ for every pair; all must be zero.

4. **Units and energy zero.** Energies in eV, on the same scale as `mu`: at $V = 0$ the levels below μ are filled.

5. **Static and non-interacting.** H must not depend on energy or on the SCF iteration. The interaction enters only through the Hubbard U of each site (Section 4), so do *not* put Hubbard or Hartree terms into H: they would be counted twice. Fixed external terms (a gate potential, a magnetic field $\mathbf{B}\cdot\boldsymbol{\sigma}$, any form of spin–orbit coupling) are fine.

6. **One orbital per site.** Each site carries one orbital with two spin states, and U acts only between the two spins of that orbital. A molecule with several orbitals per atom can still be described by treating every orbital as a "site" with its own U (`hubbard_file`). The interaction between different orbitals is then neglected.

7. **Contacts.** The left lead couples to site 1 (indices 1, 2), the right lead to site N_{sites} (indices $N_{\mathrm{orb}}-1$, N_{orb}). Number your sites so that the contacted ones come first and last, or change the two lines that set `GammaL` and `GammaR` in `src/quichotte.f90`.

Anything else is allowed. Hoppings may connect any pair of sites and have any range, on-site energies may differ, spin-flip terms may appear anywhere, and the matrix may be dense. The second-order self-energy is computed for every pair of sites, so it does not assume any particular connectivity. The energy grid needs no change: it is built from the eigenvalues of whatever H you supply.

9.3 Same terms, different shape: `hamiltonian = xyz`

If your molecule has the same kind of terms as the helix (nearest-neighbour hopping `t_hop`, and SOC `lamb` between sites i and $i+2$ built from the bond directions) but a different shape, no code is needed. Write the site positions, in chain order, to an `.xyz` file (first line: number of atoms; second line: a comment; then one `element x y z` line per site), and set `hamiltonian = xyz` and `xyz_file = my_molecule.xyz`. The number of sites is taken from the file. `tools/helix_xyz.f90` writes the helix of an input file in this format, which is a good starting point. Given exactly the helix positions, this model reproduces the helix results; this is one of the tests.

9.4 Several models of your own

The `custom` slot is the quickest way to try a model. To keep several models side by side, give each one its own routine in `src/hamiltonian.f90` and register it with a single line in `build_hamiltonian`:

```
case ('ssh');  call build_ssh_hamiltonian()
```

Then select it in `input.dat` with `hamiltonian = ssh` and `N_sites = ...`. Nothing else needs to change. The input reader accepts any model name and takes its size from `N_sites`, and `build_hamiltonian` stops with an error if it is given a name it does not know. Parameters that the new model needs are read in `read_input`, as in step 1 above.

10 Code structure

file	module	content
`src/quichotte.f90`	`(program)`	set-up, energy grid, voltage loop, output files
`src/parameters.f90`	`Parameters`	run parameters, sizes, Pauli matrices, unit numbers
`src/hamiltonian.f90`	`Hamiltonian`	helix geometry, SOC, H (analytic or from `.xyz`)
`src/green_functions.f90`	`GreenFunctions`	storage of $G_{\rm in}$, $G_{\rm out}$, Γ, grid; G_0; $G^<(t{=}0)$
`src/self_energies.f90`	`SelfEnergies`	$\Sigma^{(1)}$, $\Sigma^{(2)}$ by FFT
`src/scf.f90`	`SCF`	SCF driver, residual, Dyson/Keldysh at one energy
`src/observables.f90`	`Observables`	current, spin texture, spectral function
`src/input_output.f90`	`InputOutput`	reads `input.dat`, per-site U, Hamiltonian printout
`common/constants.f90`	`Constants`	`dp`, $\hbar$, k_B, π, i
`common/linear_algebra.f90`	`LinearAlgebra`	LAPACK wrappers, packing of G for Broyden
`common/input_parser.f90`	`InputParser`	`key = value` reader with checks
`common/broyden_mixing.f90`	`BroydenMixing`	modified Broyden solver
`common/scf_checks.f90`	`ScfChecks`	occupation / spectral-weight checks

Call sequence of one run. `read_input` → `build_helix_hamiltonian` → `hermitian_eigensystem` (grid) → for each V: `solve_scf` → `broyden_solve` → repeatedly `scf_residual` → [`equal_time_lesser`, `sigma2_compute`, `interacting_gf` for every ω] → `Current`, `spin_texture`, `write_spectral`.

Naming. $G_{\rm in}$ = `G_in` is the input of the current SCF step and $G_{\rm out}$ = `G_out` its output; components `%r`, `%a`, `%L`, `%G`, each `(Norb, Norb, n_omega)`.

Shared code. The files in `common/` are identical in QUICHOTTE and SANCHO; `make check-common` (also run by CI) fails if they differ. Change them in one place, then copy them to the other.

10.1 Common changes

A new input key
Add the variable (`parameters.f90`, or the module that uses it), add one `call input_get('key', var[, default=...])` in `read_input`, add a check in `check_input` if needed, and add it to the example input and to Section 6.

A different geometry or model
Write the positions to an `.xyz` file and call `build_hamiltonian_from_xyz` instead of `build_helix_hamiltonian` in `quichotte.f90`. For a different model altogether, see Section 9.

A sweep over a Hamiltonian parameter
Set it inside the voltage loop and call `build_helix_hamiltonian()` again (marked in the source).

A new self-energy
Compute it in `self_energies.f90` for all energies and add it in `sigma_retarded` / `sigma_lesser`.

11 Testing and validation

`make test` builds the Fortran test runner (`tests/run_tests.f90`) and runs every case listed in `tests/cases.txt`. Each case is run in `tests/work/<case>/`, and its output is compared number by number with `tests/cases/<case>/reference/`: $|x_{\mathrm{new}} - x_{\mathrm{ref}}| \le 10^{-6} + 10^{-5}|x_{\mathrm{ref}}|$ (override with the environment variables `TOL_ABS`, `TOL_REL`).

case	what it covers
`order0_noninteracting`	4-site helix, $U = 0$
`order1_hartree_fock`	first-order Σ
`order2_second_order`	FFT second-order Σ, Broyden
`order2_lefthanded_siteU`	$h = -1$, site-dependent U from `hubbard_file`
`legacy_inputSOC`	old file name `inputSOC.dat` and old key names

Run `make test` before every commit; CI also runs it on every push. After a *deliberate* change of the physics, check the new numbers, run `make update-references`, and commit the new references together with the change. `make debug` builds with bounds checking and floating-point trapping, and should pass the same tests.

Validation history. The restructured code reproduces the previous version (`SCF_stype`) to better than 3×10^{-8} in all currents, populations and $\langle S_z \rangle$ for orders 0–2 (October 2026). Before the clean-up, according to its commit history, the code agreed with the spinless code and with earlier independent implementations, gave $J = 0$ at $V = 0$, and satisfied $G^{>} = G^{<} + G^{r} - G^{a}$ with interactions.

12 Known limitations

- Perturbation theory in U up to second order; large $U/|t|$ may not converge or may converge to the unphysical solutions that the sanity checks reject.

- Wide-band leads coupled to the two end sites only; the bias is applied to the left lead.
- The energy grid is uniform; resolving sharp resonances needs a small `delta` everywhere.
- Voltages are swept upwards only (`delv` > 0).
- `build_hamiltonian_from_xyz` keeps the chain connectivity (bonds i–i+1, SOC i–i+2) of the helix.